Cash Flow Minimizer

Quiz 2 Report | Design and Analysis of Algorithms

Course	Design and Analysis of Algorithms (DAA)
Student	Naufal Dariskarim — 5025231027

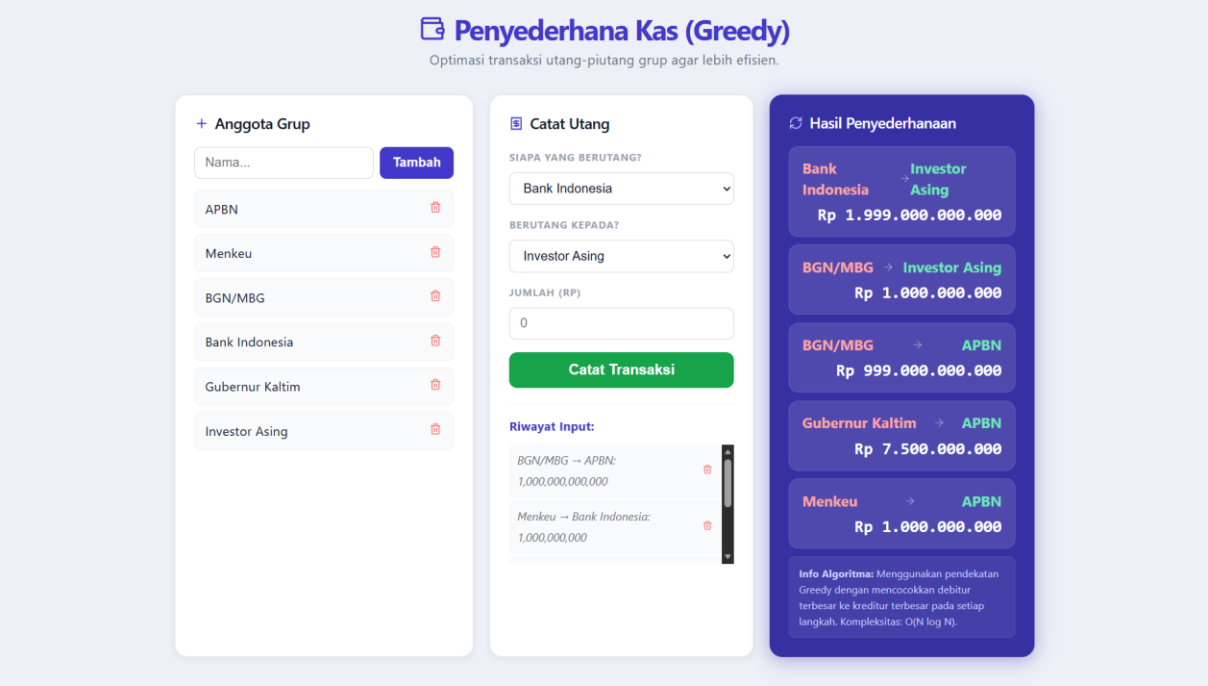

Figure 1. Cash Flow Minimizer interface and settlement results

1. Program Overview

Cash Flow Minimizer is a web-based application that solves the debt-settlement problem among a group of people. Given a set of arbitrary transactions, the program computes the minimum number of transactions required to settle all debts completely.

The implementation uses a Greedy Algorithm that repeatedly matches the person with the highest debt against the person with the highest credit, reducing the total number of required payments.

2. Program Functionality

Feature	Description
Add Member	Add individuals to the group
Remove Member	Remove a member and all associated transactions
Record Transaction	Input a payment record: who paid for whom, and the amount
Delete Transaction	Remove a specific transaction from history
Simplify Debts	Automatically compute the minimal set of settlements using the Greedy algorithm

Core Algorithm - Greedy Cash Flow Minimization

1. Calculate Net Balance - For each person, compute total received minus total paid. A positive balance means they are owed money; a negative balance means they owe money.
2. Separate Creditors and Debtors - Split all members into two sorted lists based on their net balance in descending order.
3. Greedy Match - At each step, pair the largest debtor with the largest creditor, transfer the minimum amount between them, and reduce both balances accordingly.
4. Repeat until all balances are settled.

Time Complexity: O(N log N) - dominated by the sorting step.

3. Program Flow

User actions update application state, and the simplified settlement is derived automatically whenever the relevant state changes.

- Add or remove members to update the member list state.
- Record a transaction (from, to, amount) to append it to the transactions list.
- Run the greedy settlement logic on every change to produce simplified transactions.
- Render the simplified transactions on the UI.

State Flow (React)

members[] and transactions[] feed useMemo(), which produces simplifiedTx[] and then renders it on the UI.

4. Technologies Used

Technology	Role
React 19	UI framework - component-based rendering and state management
Vite 8	Build tool and local development server with Hot Module Replacement (HMR)
JavaScript (ES2024)	Core programming language
JSX	HTML-in-JS syntax extension for React components
Lucide React	Icon library (Wallet, Receipt, RefreshCw, etc.)
CSS-in-JS (inline styles)	Component styling via JavaScript style objects
Node.js + npm	Package manager and runtime environment

No external CSS frameworks or backend services are used. The entire algorithm runs client-side in the browser.

5. How to Run Locally

Prerequisites

- Node.js (v18 or higher)
- npm (comes bundled with Node.js)

Steps

5. Navigate to the project directory: cd "Q2_DAA"
6. Install dependencies: npm install
7. Start the development server: npm run dev
8. Open http://localhost:5173/ in your browser.

The terminal will display a Vite ready message with the local development URL.

Stopping the Server

Press Ctrl + C in the terminal to stop the dev server.

Build for Production

Run npm run build. The output will be placed in the dist/ folder, ready for static hosting.

6. Project Structure

- public/ - Static assets
- src/App.jsx - Main component containing all logic and UI
- src/App.css - Additional styles
- src/main.jsx - React entry point
- src/index.css - Global CSS reset
- index.html - HTML shell
- package.json - Project metadata and dependencies
- vite.config.js - Vite configuration
- Report.md - Source report file